



Лекция 7. Рекуррентные сети и трансформеры

Архитектуры для последовательностей и текста — от RNN до современных гибридных моделей 2025–2026 годов.

Место лекции в курсе и цели

Двухслойная структура

Слой 1 — как устроены модели последовательностей от RNN до современных гибридных и разреженных архитектур 2025–2026 годов.

Слой 2 — какие свойства этих моделей превращаются в поверхность атаки.

Шесть целей лекции

1. Формально записать задачу моделирования последовательности и авторегрессионную факторизацию.
2. Понять, почему рекуррентность плохо масштабируется и как внимание снимает узкое место.
3. Вывести и посчитать self-attention, multi-head attention, KV-кэш.
4. Знать современный стек decoder-only модели: RoPE, RMSNorm, pre-LN, SwiGLU, GQA/MLA, MoE, FlashAttention.
5. Понимать возврат рекуррентности: линейное внимание, SSM (Mamba/Mamba-2/Mamba-3), гибриды $\approx 3:1$.
6. Понимать конвейер «предобучение → SFT → выравнивание → инференс → RAG/агент» и знать, где возникают уязвимости.

Задача моделирования последовательностей

Постановка задачи

Дана последовательность токенов $\mathbf{x} = (x_1, \dots, x_T)$, где $x_i \in V$,
 V — размер словаря. Языковая модель задаёт распределение $p_\theta(x)$.
Авторегрессионная факторизация по цепному правилу:

$$p_\theta(x_1, \dots, x_T) = \prod_{t=1}^T p_\theta(x_t | x_{<t})$$

Обучение — минимизация кросс-энтропии (negative log-likelihood):

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t | x_{<t}).$$

Стандартная метрика качества — **перплексия**:

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t | x_{<t})\right)$$

Перплексию удобно трактовать как «эффективное число равновероятных вариантов» на каждом шаге: $\text{PPL} = V$ означает полное незнание, $\text{PPL} \rightarrow 1$ — детерминированное предсказание.

Teacher forcing и exposure bias

При обучении модель на шаге t видит **истинный префикс** x (exposure bias) означает, что поведение модели на «нетипичных» префиксах, которых не было в обучении, слабо контролируется.



▸ Атака на LLM — это поиск входного префикса s , при котором $p_\theta(y_{\text{target}} | \text{prompt} \oplus s)$ велика.

Все атаки сводятся к максимизации этой вероятности: вручную (jailbreak), градиентом (adversarial suffixes) или через внешние данные (prompt injection).

Типы задач над последовательностями

Тип	Вход → выход	Примеры	Архитектура
many-to-one	последовательность → метка	анализ тональности, детекция спама	encoder-only
many-to-many, выровненные	$T \rightarrow T$	POS-разметка, NER	encoder-only
seq2seq, невыровненные	$T \rightarrow T'$	перевод, суммаризация	encoder-decoder
авторегрессионная генерация	префикс → продолжение	чат, код, агенты	decoder-only
ранжирование/поиск	текст → вектор	RAG, семантический поиск	encoder + contrastive

Токенизация: первая поверхность атаки

Символы/байты

Словарь 256. Нет OOV (Out-of-Vocabulary), устойчив к опечаткам. Минус: длинные последовательности, слабая семантика на токен.

Слова

Словарь 10^5 – 10^6 . Семантически осмысленные единицы. Минус: OOV, огромная выходная матрица, плохо для морфологически богатых языков.

Подслова (BPE)

Словарь $3 \cdot 10^4$ – $2 \cdot 10^5$. Компромисс, нет OOV при byte-level. Минус: границы токенов неинтуитивны.

Байтовые патчи (BLT)

Нет фиксированного словаря. Адаптивная длина патча, устойчивость к шуму. Byte Latent Transformer (ACL 2025): патчи масштабируются лучше токенов.

Byte-Pair Encoding итеративно склеивает самую частую пару соседних единиц: начиная с символов/байтов, выполняется K слияний, где K задаёт итоговый размер словаря. Byte-level BPE (GPT-2 и далее) работает над байтами UTF-8, что исключает OOV принципиально.



Специальные токены, шаблон диалога и глитч-токены

Шаблон диалога

Современная чат-модель видит не «сырой текст», а размеченную структуру:

```
<|system|> Ты полезный ассистент. <|end|>  
<|user|> Переведи текст ниже. <|end|>  
<|assistant|>
```

Ключевой факт: роли system, user, assistant — это обычные токены в одном и том же потоке, а не отдельные защищённые каналы. Никакого аппаратного или архитектурного разделения привилегий нет; приоритет ролей моделируется только статистически, обучением (The Instruction Hierarchy, arXiv 2404.13208).

Артефакты токенизатора

Часть токенов словаря почти не встречается в обучающем корпусе. Их эмбединги остаются почти необученными, и модель на них ведёт себя аномально: отказывается воспроизводить строку, галлюцинирует, ломает разметку. Такие «глитч-токены» детектируются автоматически по свойствам входной и выходной матриц эмбедингов (Fishing for Magikarp, arXiv 2405.05417; GlitchMiner).

- ▢ ▶ нестандартные токены — готовая «дырка»; инъекция строки вида <|system|> — базовый приём захвата роли; кодировки (base64, leetspeak) меняют разбиение на токены — механизм mismatched generalization.

Рекуррентные нейронные сети

Базовая RNN (Elman)

$$h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b_h),$$

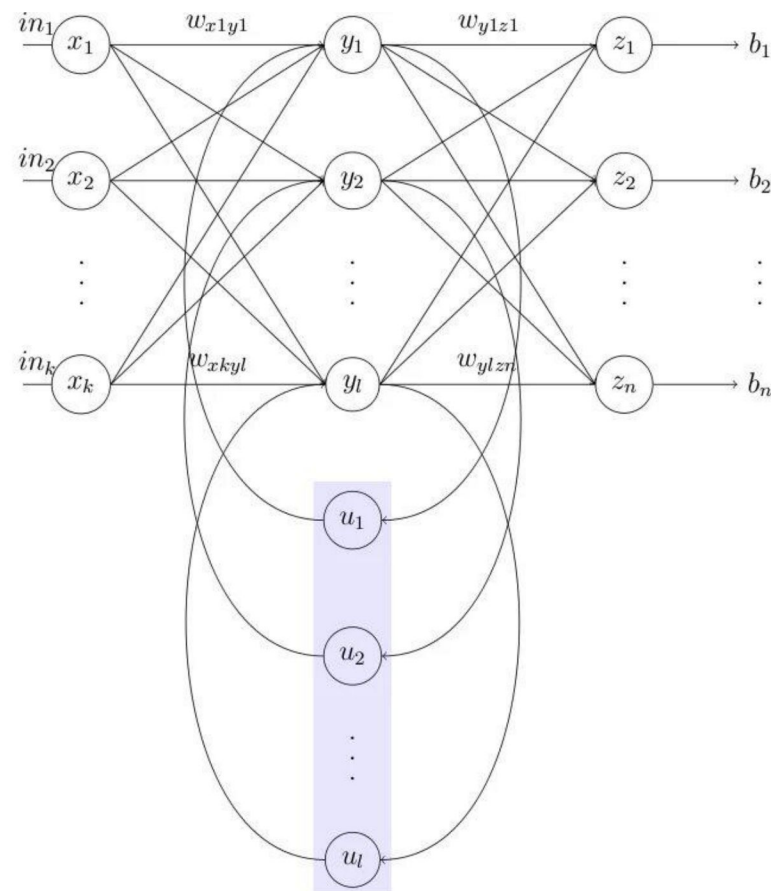
$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y).$$

Число параметров не зависит от длины входа; h_t — сжатое представление всего префикса; вычисление принципиально последовательно по t .

Проблема градиента (BPTT)

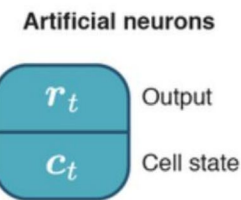
Градиент по h_t через k шагов содержит произведение якобианов. Если спектральный радиус $\rho(W_{hh}) < 1$ — **исчезающий градиент**, дальние зависимости не обучаются. Если $\rho(W_{hh}) > 1$ — **взрывающийся градиент**, обучение расходится. Практические средства: обрезка градиента по норме, ортогональная инициализация, truncated BPTT, вентильные архитектуры.

$$\frac{\partial h_T}{\partial h_{T-k}} = \prod_{j=T-k+1}^T \text{diag}(\phi'(\cdot)) W_{hh}^T.$$

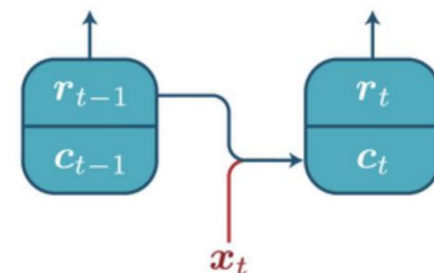


A

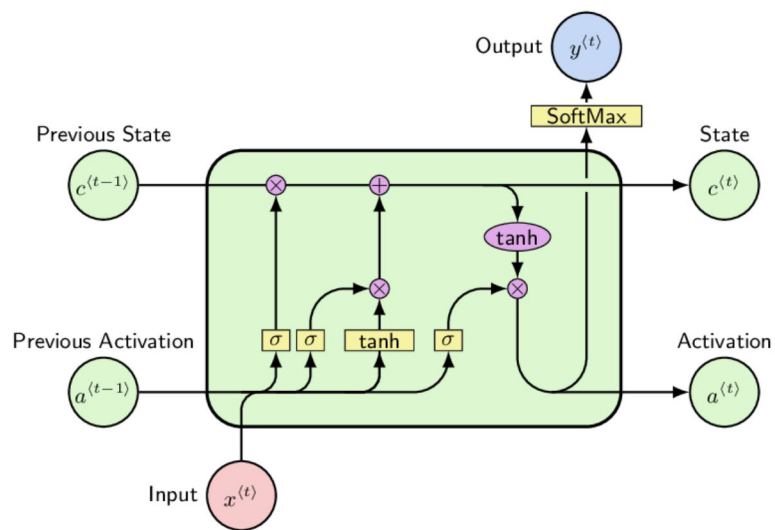
Linear operation
→
Multiplicative gating
□
Identity connection



Vanilla/Elman RNN



Рекуррентные нейронные сети



LSTM: «карусель постоянной ошибки»

Долгая краткосрочная память вводит отдельное состояние ячейки c_t с **аддитивным обновлением**: при $f_t \approx 1$ градиент по c_t проходит назад почти без затухания, поскольку путь аддитивный, а не мультипликативный. Три вентиля: забывания f_t , входа i_t , выхода o_t .

GRU: упрощение

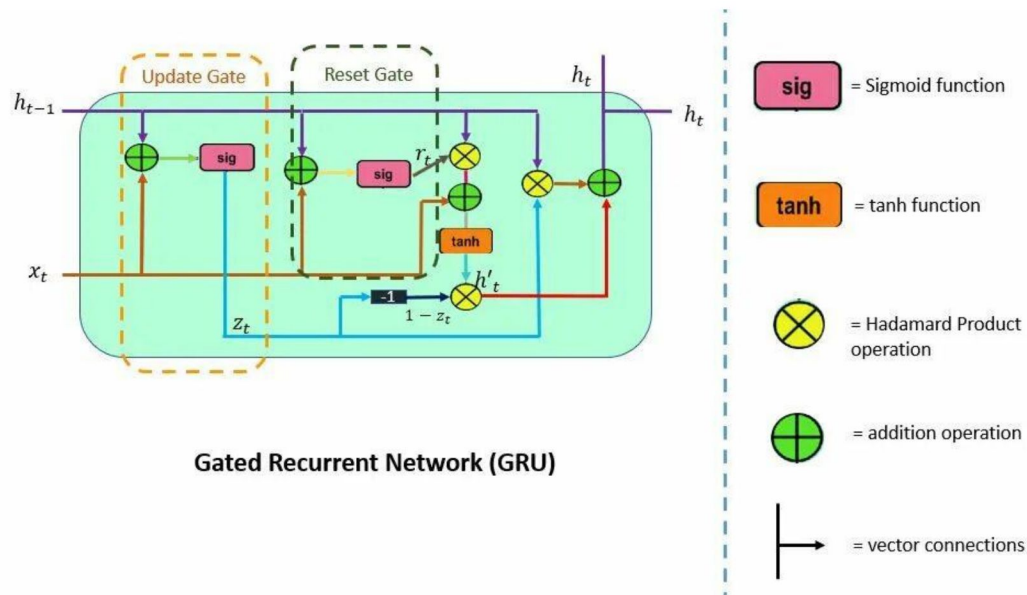
Два вентиля и одно состояние. GRU обычно на 20–30% дешевле LSTM по параметрам при сопоставимом качестве на средних объёмах данных (Cho et al., arXiv 1406.1078).

Двунаправленные и глубокие RNN

Двунаправленная RNN конкатенирует прямое и обратное состояния:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

Годится для разметки и классификации, **не годится** для авторегрессионной генерации: обратный проход требует будущих токенов.



Gated Recurrent Network (GRU)

Seq2seq и bottleneck

Модель кодировщик-декодировщик сжимает весь вход в один вектор $c = h_T^{\text{enc}}$ (Sutskever et al., arXiv 1409.3215). Качество перевода падает с ростом длины предложения: один вектор фиксированной размерности не удерживает всю информацию. Это узкое место стало прямой мотивацией механизма внимания.

Аддитивное внимание Багданау

Вместо одного вектора c декодировщик на каждом шаге i строит свой контекст:

$$e_{ij} = a(s_{i-1}, h_j), \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^T \exp(e_{ik})}, c_i = \sum_{j=1}^T \alpha_{ij} h_j,$$

где a — небольшая обучаемая сеть (Bahdanau et al., arXiv 1409.0473). Мультипликативный вариант Луонга использует:

$$e_{ij} = s_{i-1}^\top W h_j$$

Интерпретация: мягкая ассоциативная память

Внимание — это дифференцируемый lookup по таблице «ключ → значение»: запрос сравнивается со всеми ключами, а результат — взвешенная сумма значений.

- ▢ ▶ любой фрагмент контекста, чей ключ окажется похож на запрос, повлияет на выход независимо от его происхождения и доверенности. Это архитектурная первопричина indirect prompt injection (Greshake et al., arXiv 2302.12173).

Трансформер: Attention Is All You Need

Архитектура предложена в работе «Attention Is All You Need» (Vaswani et al., 2017): рекуррентность убрана полностью, остаются внимание, полносвязные блоки, нормализация и остаточные связи.

Scaled dot-product attention

Для $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK}{\sqrt{d_k}} + M\right)V$$

Делитель $\sqrt{d_k}$ нужен: при независимых компонентах с единичной дисперсией $\text{Var}(q^T k) = d_k$; без нормировки логиты растут как d_k , softmax насыщается, градиент исчезает. Матрица масок M реализует каузальную маску ($M_{ij} = -\infty$ при $j > i$) и маску padding-токенов.

Многоголовое внимание (МНА)

$$\text{MHA}(X) = [\text{head}_1; \dots; \text{head}_H]W^O$$

$$\text{head}_h = \text{Attention}(XW_h^Q, XW_h^K, XW_h^V)$$

Обычно $d_k = d_v = d/H$, так что стоимость МНА сопоставима с одноголовым вниманием размерности d .

Разные головы специализируются: синтаксические связи, позиционные шаблоны, «индукционные головы» (копирование по шаблону «A B ... A → B»), головы поиска в контексте.

Pre-LN блок (современный вариант)

Pre-LN (нормализация до подслоя) даёт устойчивое обучение глубоких стеков без разогрева-«костылей», в отличие от исходного post-LN.

Современный стек decoder-only: SwiGLU, RMSNorm, RoPE

SwiGLU вместо ReLU FFN

Классический FFN: $\text{FFN}(x) = W_2 \text{ReLU}(W_1 x)$, расширение $d \rightarrow 4d \rightarrow d$. Современный вариант — SwiGLU:

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}(xW_1) \odot xW_3)W_2,$$

При равном числе параметров даёт лучшую перплексию (GLU Variants Improve Transformer, arXiv 2002.05202).

RMSNorm

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \varepsilon}} \odot g,$$

Без вычитания среднего и без сдвига — дешевле и численно устойчивее LayerNorm (Zhang & Sennrich, arXiv 1910.07467).

Позиционное кодирование: сравнение подходов

Self-attention перестановочно инвариантно: без позиционных признаков «собака укусила человека» и «человек укусил собаку» дают одинаковые представления.

Подход	Идея	Экстраполяция
Синусоидальное	$PE_{t,2i} = \sin(t/10000^{2i/d})$	слабая
Обучаемые абсолютные	таблица позиций	нет
RoPE	поворот пар координат q, k на угол $\theta_i t$	хорошая (NTK-scaling, YaRN)
ALiBi	линейный штраф $-m_h(i-j)$ к логитам	«train short, test long»
NoPE	без явного кодирования	ограниченная

RoPE — де-факто стандарт decoder-only моделей (RoFormer, arXiv 2104.09864).

Три семейства предобученных моделей

Encoder-only

Маска: полная (двусторонняя). **Цель:** masked LM — восстановить 15% замаскированных токенов. **Модели:** BERT, RoBERTa, DeBERTa. **Сильная сторона:** классификация, NER, эмбединги для поиска.

Decoder-only

Маска: каузальная. **Цель:** предсказание следующего токена. **Модели:** GPT-3, Llama, Mistral, Qwen, DeepSeek. **Сильная сторона:** генерация, in-context learning, агентность.

Encoder-decoder

Маска: полная в энкодере, каузальная в декодере + кросс-внимание. **Цель:** denoising / span corruption. **Модели:** T5, BART. **Сильная сторона:** перевод, суммаризация, структурные преобразования.

Почему победила decoder-only схема

Одна цель обучения (next-token prediction) масштабируется на произвольные корпуса без разметки; любая задача переформулируется как продолжение текста; появляется **in-context learning**: модель решает задачу по примерам в промпте без изменения весов (Brown et al., arXiv 2005.14165).

- ▢ in-context learning — одновременно главная функциональность и главная уязвимость. Many-shot jailbreaking прямо эксплуатирует эту способность, заполняя длинный контекст десятками фиктивных примеров «вредных» диалогов (Anthropic, 2024).

Законы масштабирования и эмерджентность



Законы масштабирования

Потери убывают степенно по числу параметров, объёму данных и вычислениям. При фиксированном бюджете существует оптимальное соотношение размера модели и объёма данных (линия Kaplan → Chinchilla). Свойства модели, включая устойчивость и «выравненность», зависят не только от архитектуры, но и от режима обучения.



Эмерджентность

Часть способностей (арифметика по шагам, следование инструкциям, использование инструментов) проявляется только выше определённого масштаба. Аналогично масштабируются и уязвимости — например, чувствительность к длинному контексту.



Контекстные представления

В отличие от статических word2vec/GloVe, представление токена в трансформере зависит от всего контекста, поэтому «смысл» токена управляем извне — через текст. Это фундаментальное свойство, которое делает атаки через контекст системными.

KV-кэш: эффективный инференс

Принцип и формула

При авторегрессионной генерации ключи и значения предыдущих токенов не меняются, поэтому их кэшируют. Память кэша:

$$\text{Mem}_{KV} = 2 \cdot L \cdot H_{kv} \cdot d_{\text{head}} \cdot n \cdot b$$

где L — число слоёв, H_kv — число KV-голов, n — длина контекста, b — размер элемента в байтах.

Численный пример (70B модель)

L = 80, d_head = 128, fp16 (b = 2):

- Полное MHA, $H_{kv} = 64$: $2 \cdot 80 \cdot 64 \cdot 128 \cdot 2 = 2\,621\,440$ байт ≈ 2.5 Мб на один токен; при 32 768 токенах ≈ 80 Гб — больше, чем сами веса.
- GQA с H_kv = 8: 320 Кб на токен, при 32 768 токенах ≈ 10 Гб — в 8 раз меньше.

Механизмы экономии на KV

Механизм	Идея	Эффект
MQA	одна KV-голова на все Q-головы	максимальная экономия, потеря качества
GQA	G групп Q-голов на одну KV-голову	качество близко к MHA, скорость близка к MQA
MLA	низкоранговое латентное сжатие K и V	сильное сжатие при сохранении качества (DeepSeek-V2)
Скользящее окно	внимание в пределах окна w	линейная память, ограниченный горизонт
DSA	обучаемая разреженная выборка позиций	снижение сложности (DeepSeek-V3.2)

FlashAttention и патологии длинного контекста

FlashAttention

Точное внимание, переписанное как IO-эффективный алгоритм: **тайлинг по блокам**, онлайн-softmax, отказ от материализации матрицы $n \times n$ в медленной памяти. Третья версия использует асинхронность и низкую точность на современных ускорителях (FlashAttention-3, arXiv 2407.08608).

Важно: это оптимизация реализации, а не изменение математики модели. Результат идентичен стандартному вниманию.

«Lost in the Middle»

Заявленное окно контекста не означает равномерного использования. Модели демонстрируют **U-образный профиль**: информация в начале и конце контекста используется существенно лучше, чем в середине (Liu et al., arXiv 2307.03172).

- ▢ ► позиция вредоносной вставки в документе влияет на успех инъекции; длинный контекст даёт «объём» для many-shot атак; кэширование префикса создаёт риски cache-poisoning.

Смесь экспертов (MoE)

Принцип

Идея: заменить один плотный FFN набором N экспертов и на каждый токен активировать только $k \ll N$ из них :

$$y = \sum_{i \in \text{TopK}(g(x))} g_i(x) E_i(x), g(x) = \text{softmax}(W_g x).$$

Так растёт общее число параметров при почти неизменной стоимости токена.

Ключевые вехи

- **Switch Transformer**: top-1 маршрутизация, балансировка нагрузки (arXiv 2101.03961).
- **Mixtral 8×7B**: 8 экспертов, 2 активны, ~47 млрд общих против ~13 млрд активных параметров (arXiv 2401.04088).
- **DeepSeek-V3**: ~671 млрд общих, ~37 млрд активных на токен, MLA + тонкогранулярные эксперты с общими «shared» экспертами.

Инженерные проблемы MoE

- Балансировка нагрузки между экспертами
- Устойчивость маршрутизации
- Стоимость коммуникации при экспертном параллелизме
- «Мёртвые» эксперты



► маршрутизация зависит от входа — сама является объектом атаки. Подобранный вход способен систематически направлять токены к «неудачным» экспертам. Разница между общими и активными параметрами означает, что безопасное поведение может быть закодировано неравномерно по экспертам.

Возврат рекуррентности: линейное внимание и SSM

Почему тема снова открыта

Трансформер платит квадратичным prefill и линейно растущим KV-кэшем. Рекуррентная модель имеет постоянное состояние и линейный по длине проход, но сжимает прошлое, а не хранит его. Основная линия развития 2023–2026 годов — соединить оба свойства.

Линейное внимание

Замена softmax на разложимое ядро $\text{sim}(q, k) = \phi(q)^\top \phi(k)$ позволяет переставить умножения:

$$\text{out}_i = \frac{\phi(q_i)^\top \sum_{j \leq i} \phi(k_j) v_j^\top}{\phi(q_i)^\top \sum_{j \leq i} \phi(k_j)}.$$

Суммы накапливаются рекуррентно — модель эквивалентна RNN с матричным состоянием $S_i = \sum_{j \leq i} \phi(k_j) v_j^\top$, память не зависит от n . **Цена** — потеря избирательного «точечного» доступа к отдельному давнему токenu.

Модели пространства состояний (SSM)

Непрерывная линейная система $h'(t) = Ah(t) + Bu(t), y(t) = Ch(t)$ после дискретизации даёт рекуррентность $h_k = \bar{A}h_{k-1} + \bar{B}u_k$, которую можно вычислять и как свёртку (параллельное обучение), и как рекуррентность (константный инференс).

- **Mamba**: параметры $\bar{A}, \bar{B}$ зависят от входа (selective SSM), линейное время (arXiv 2312.00752).
- **Mamba-2**: двойственность «SSM $\leftrightarrow$ структурированное маскированное внимание» (SSD), более быстрые матричные алгоритмы.
- **Mamba-3 (2026)**: комплекснозначное обновление состояния, MIMO-формулировка; +0.6 п.п. к Gated DeltaNet на 1.5B, +1.2 п.п. от MIMO-варианта (arXiv 2603.15569).
- **RWKV**: рекуррентная формулировка с параллельным обучением (arXiv 2305.13048).
- **Titans**: обучаемый модуль памяти, обновляемый во время инференса (arXiv 2501.00663).

Гибридные архитектуры: доминирующий паттерн 2026 года

Практика сошлась на чередовании: большинство слоёв — рекуррентные/линейные, каждый k-й слой — полное softmax-внимание для точного доступа к отдельным токенам. Типичное соотношение — **≈3:1**.

Модель	Рекуррентный механизм	Полное внимание	Конфигурация
Qwen3-Next 80B-A3B	Gated DeltaNet	gated attention	36 рекуррентных + 12 attention, 3:1
Qwen3.5 397B-A17B	рекуррентные слои	attention	60 слоёв, 15 повторов группы 3:1
Kimi Linear 48B-A3B	Kimi Delta Attention	gated MLA	20 KDA + 7 MLA, ≈3:1
Ling 2.5	Lightning Attention	MLA	1 MLA на 7 линейных
Nemotron 3 Nano 30B	Mamba-2	attention	23 Mamba-2 + 23 MoE + 6 attention
Jamba	Mamba	attention	чередование + MoE (arXiv 2403.19887)

▢ ▶ суффикс, подобранный на плотном трансформере, может по-разному вести себя на гибриде со сжатым состоянием. Сжатое состояние даёт новые уязвимости — «затирание» ранней системной инструкции длинным потоком последующего текста.

Сравнение архитектур: RNN vs Трансформер vs SSM vs Гибрид

Свойство	RNN/LSTM	Трансформер	Линейное внимание / SSM	Гибрид
Обучение	последовательное	параллельное	параллельное (свёрточная форма)	параллельное
Стоимость prefill	$O(n)$	$O(n^2)$	$O(n)$	почти $O(n)$
Память при декодировании	$O(1)$	$O(n)$ KV-кэш	$O(1)$	растёт только на attention-слоях
Точный доступ к давнему токenu	нет	да	ограниченно	да, через редкие attention-слои
Длина пути между зависимостями	$O(n)$	$O(1)$	$O(1)$ со сжатием	$O(1)$

Альтернативные парадигмы генерации

Диффузионные языковые модели

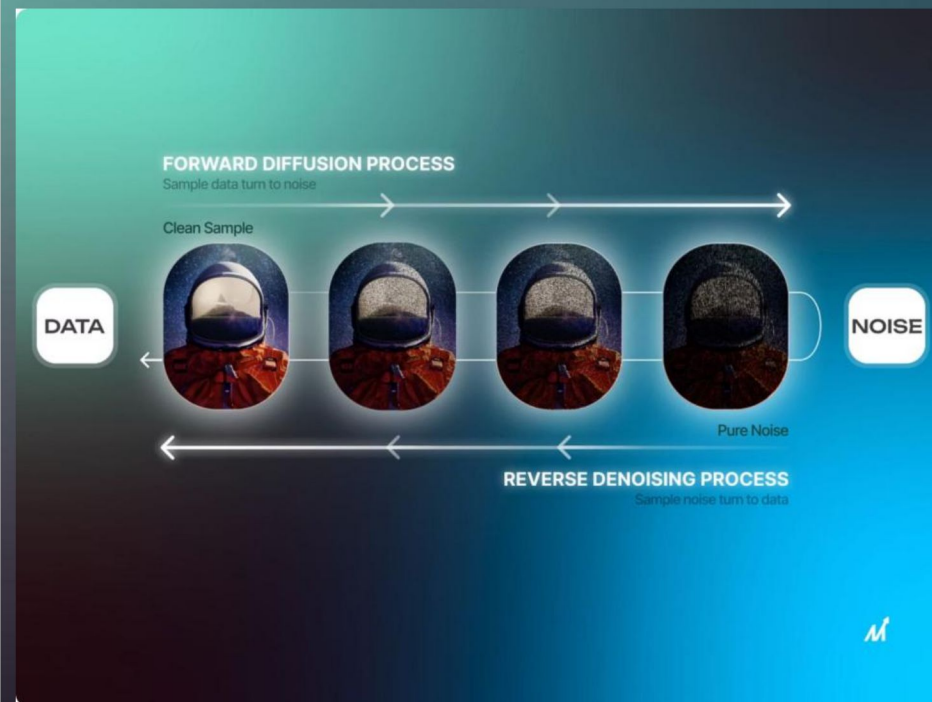
Генерация не слева направо, а итеративным «расшумлением» замаскированной последовательности. Заявлена конкурентоспособность с авторегрессионными моделями сопоставимого масштаба (LLaDA, arXiv 2502.09992; Dream 7B, 2025). Для безопасности существенно: понятие «префикса» размывается, а значит меняются механика префилл-атак и точки контроля вывода.

Tokenizer-free и байтовые патчи

Byte Latent Transformer (BLT, ACL 2025): исчезает часть атак, связанных с артефактами словаря, но появляется чувствительность к байтовым шаблонам. Адаптивная длина патча масштабируется лучше фиксированных токенов.

Мультимодальные входы

Изображение/аудио проецируются в то же пространство токенов, поэтому инструкция может быть напечатана на картинке — прямой мост от физических атак лекции 6 к инъекциям лекции 8.



Декодирование: как из логитов получается текст

Распределение с температурой

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

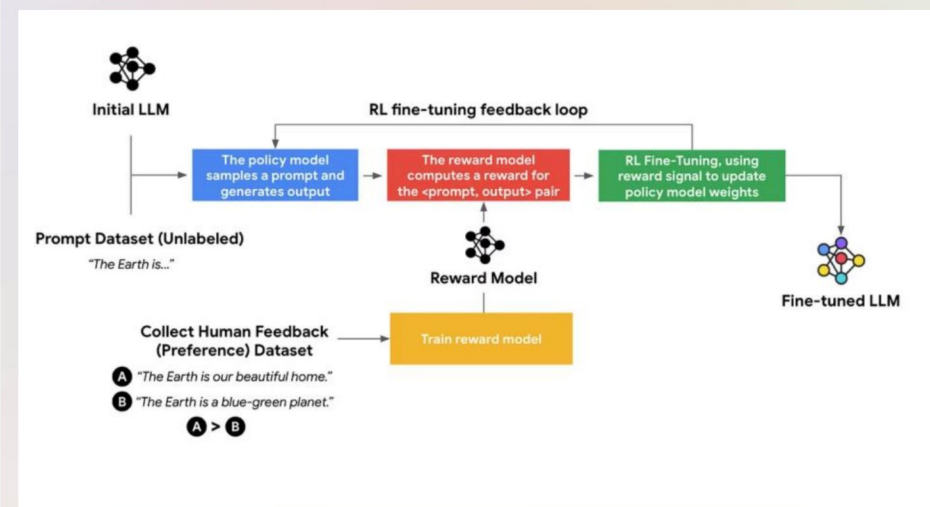
Стратегия	Правило	Свойства
greedy	$\operatorname{argmax} p_i$	детерминированно, повторы
beam search	В лучших гипотез по log-prob	хорошо для перевода, «скучно» для чата
top-k	сэмплирование из k вероятных	контроль хвоста
top-p (nucleus)	минимальное множество с массой $\geq p$	адаптивный размер хвоста
штрафы за повтор	модификация логитов	борьба с деградацией

Инженерия инференса

Спекулятивное декодирование: черновая модель предлагает γ токенов, целевая проверяет их одним проходом — ускорение без изменения распределения (Leviathan et al., arXiv 2211.17192). Также: квантизация (INT8/INT4), непрерывный батчинг, paged-KV.

- ❑ (1) стохастичность делает атаку вероятностным событием — best-of-n повышает шанс обхода фильтра;
- (2) сравнение результатов при разных T , p некорректно;
- (3) квантизация и спекулятивное декодирование могут ослаблять выравнивание.

Конвейер адаптации и выравнивания



RLHF, DPO и обучение рассуждению

RLHF

Обучает модель награды r_θ на парах предпочтений, затем оптимизирует политику с KL-ограничением к референсной модели (InstructGPT, arXiv 2203.02155).

DPO (Direct Preference Optimization)

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

Прямая оптимизация предпочтений без явной модели награды (Rafailov et al., arXiv 2305.18290).

Обучение рассуждению (RLVR)

Chain-of-thought изначально был приёмом промптинга (Wei et al., arXiv 2201.11903). Затем длинные рассуждения стали обучать напрямую подкреплением с проверяемой наградой: **DeepSeek-R1** демонстрирует появление способности к длинному рассуждению преимущественно за счёт RL (arXiv 2501.12948). Для безопасности это важно двояко: цепочка рассуждений и повышает надёжность, и создаёт новую поверхность — атаки на сам процесс рассуждения и утечки через рассуждение.

LoRA и QLoRA

Обновление веса: $W' = W + (\alpha/r)BA$, где $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$; обучаются только A , B (arXiv 2106.09685). QLoRA добавляет квантизацию базовых весов.

Насколько «глубоко» выравнивание?

Экспериментально показано, что отказ от выполнения запроса у выровненных моделей во многом опосредован **одним направлением в пространстве активаций**: его удаление снимает отказы, а добавление вызывает отказы даже на безобидных запросах (Arditi et al., arXiv 2406.11717).

Это прямое свидетельство того, что **выравнивание — сравнительно тонкий слой поверх базовой языковой модели**, а не переобучение её знаний.

Дообучение, включая LoRA на безобидных данных, способно снимать выравнивание — это отдельный класс угроз цепочки поставки моделей.

Две базовые причины успеха jailbreak

- 1 Конкурирующие цели**
Полезность против безопасности — модель оптимизирована на оба сигнала одновременно.
- 2 Рассогласованное обобщение**
Safety-обучение покрывает не все области входного распределения (Jailbroken, arXiv 2307.02483).

RAG, инструменты и агенты

Типовой конвейер RAG

Запрос → эмбединг → поиск по векторной базе → top-k фрагментов → сборка промпта → генерация. Вызов инструментов: модель порождает структурированный вызов функции, среда исполняет его и возвращает результат в тот же контекст.

Четыре части контекста агента

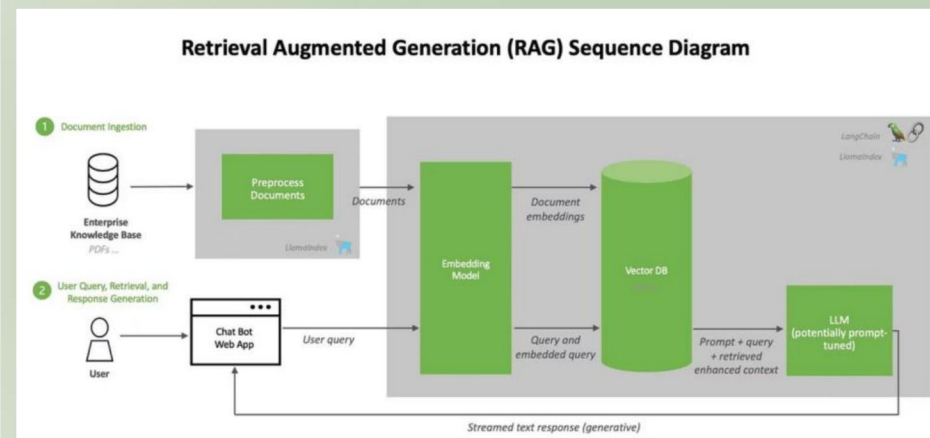
1. Системная политика разработчика (**доверенная**)
2. Запрос пользователя (**частично доверенный**)
3. Результаты поиска, страницы, письма, документы, вывод инструментов (**недоверенные**)
4. История собственных рассуждений и вызовов

Системная проблема

Архитектурно все четыре части — просто токены. Модель различает их только по статистически выученным признакам, что и делает **indirect prompt injection системной, а не «баговой» проблемой** (Greshake et al., arXiv 2302.12173).

Частичные средства защиты

- Обучение иерархии инструкций (arXiv 2404.13208)
- Изоляция инструментов
- Детерминированные проверки на выходе
- Ограничение привилегий агента

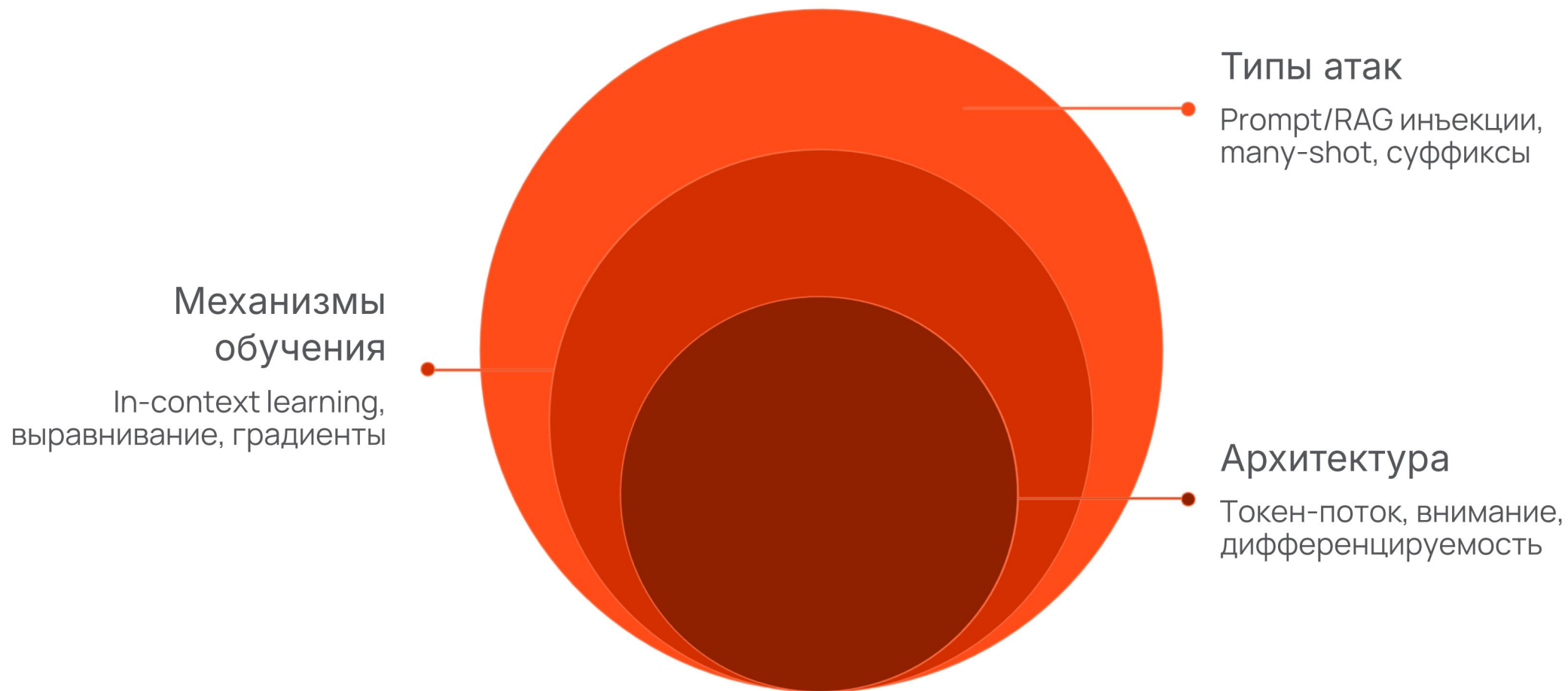


Оценка моделей

Что измеряем	Метрики и наборы	Ограничения
Языковое моделирование	перплексия	несопоставима при разных токенизаторах
Генерация	BLEU, ROUGE, chrF, judge-модели	слабая корреляция с полезностью
Понимание	GLUE/SuperGLUE, MMLU и аналоги	загрязнение обучающих данных
Длинный контекст	needle-in-a-haystack, многошаговый поиск	не проверяют равномерность внимания
Безопасность	наборы вредных запросов, ASR	зависят от судьи, декодирования и формулировок

 Методическое замечание: **ASR — не свойство модели, а свойство тройки «модель + протокол атаки + судья»**. Сравнивать числа из разных работ без совпадения протокола некорректно — ровно как нельзя сравнивать физические атаки без совпадения условий съёмки.

От архитектуры к уязвимостям



Каждое архитектурное свойство трансформера имеет прямое следствие в виде класса атак. Понимание этой связи — ключевая цель лекции 7 как подготовки к лекции 8.

Сводная таблица: свойство → атака → защита

Архитектурное свойство	Атака	Направление защиты
Инструкции и данные — один поток токенов	prompt injection, прямая и косвенная	иерархия инструкций, изоляция
Внимание как мягкий поиск по всему контексту	инъекция через RAG, веб-страницу, картинку	фильтрация источников, детерминированные проверки
In-context learning	many-shot jailbreaking, role-play атаки	ограничение длины недоверенного текста
Выравнивание — тонкий слой (одно направление отказа)	jailbreaking: конкурирующие цели, рассогласованное обобщение	adversarial training, внешние классификаторы
Модель дифференцируема по эмбедингам, вход дискретен	adversarial suffixes (GCG)	детекция перплексией, канареечные проверки
Общность представлений между моделями	универсальные и переносимые атаки	ансамблевые проверки, разнообразие фильтров
Артефакты токенизатора	глitch-токены, обфускация кодировками	нормализация входа, аудит словаря
Стохастическое декодирование	best-of-n, регенерация	фиксация T, проверка на нескольких сэмплах
Длинный контекст, «lost in the middle»	размещение инъекции в «сильной» зоне	позиционно-инвариантные проверки
Агентность: инструменты, память, файлы	эксфильтрация данных, цепочки вредных вызовов	минимальные привилегии, подтверждение действий

Математический мост к adversarial suffixes (GCG)

Задача атакующего

Найти суффикс $s = (s_1, \dots, s_k)$, $s_i \in V$, максимизирующий вероятность целевого начала ответа (например, «Sure, here is ...»):

$$\max_{s \in V^k} \log p_{\theta}(y_{\text{target}} \mid x \oplus s)$$

Прямой перебор невозможен: пространство V^k астрономическое ($32000^{20} \approx 10^{90}$).

Greedy Coordinate Gradient (GCG)

Модель дифференцируема по входным эмбедам, поэтому оценивается чувствительность к замене токена в позиции i через градиент по его one-hot представлению:

$$g_i = \nabla_{e_{s_i}} (-\log p_{\theta}(y_{\text{target}} \mid x \oplus s)) \in \mathbb{R}^{|V|}$$

Берутся k наиболее перспективных кандидатов по $-g_i$, затем точно вычисляется потеря для выборки кандидатов и принимается лучшая замена. При оптимизации по нескольким запросам и нескольким моделям получаются **универсальные и переносимые суффиксы** (Zou et al., arXiv 2307.15043).

Три вывода для понимания GCG

1 Дискретность входа

Причина, по которой нельзя применить обычный PGD из лекций о зрении: градиент даёт лишь суррогат для выбора кандидатов, а не сам шаг.

2 Дифференцируемость по эмбедам

Причина, по которой атака всё же намного эффективнее случайного поиска.

3 Общность представлений

Причина переносимости суффиксов между разными моделями — аналогично переносимости adversarial-примеров между CNN.

Контрольный список готовности к следующей лекции

01

Токенизация

Как из текста получается последовательность токенов и почему разбиение неоднозначно?

03

Недоверенный контекст

Почему модель «не знает», что фрагмент контекста пришёл из недоверенного источника?

05

Декодирование и воспроизводимость

Как параметры декодирования влияют на воспроизводимость ответа? Почему модель дифференцируема по эмбедингам, но не по токенам?

02

Системный промпт

Где физически находится системный промпт во входе модели? Что физически отделяет его от пользовательского текста?

04

Цели обучения

Что именно оптимизируется при обучении LM и что при выравнивании?

06

KV-кэш и архитектуры

Что такое KV-кэш и почему длина контекста — ресурс? Чем отличается плотный трансформер от MoE и гибрида по способу хранения прошлого?

Глоссарий ключевых терминов

Термин	Английский	Краткое определение
Токенизация	tokenization	разбиение текста на элементы словаря
Каузальная маска	causal mask	запрет внимания к будущим позициям
Многоголовое внимание	multi-head attention	параллельные подпространства внимания
KV-кэш	KV cache	сохранённые ключи и значения предыдущих токенов
Групповое внимание	grouped-query attention	несколько Q-голов на одну KV-голову
Латентное внимание	multi-head latent attention	низкоранговое сжатие K/V
Смесь экспертов	mixture of experts	разреженная активация подсетей
Модель пространства состояний	state space model	линейная рекуррентность с обучаемой динамикой
Обучение в контексте	in-context learning	решение задачи по примерам в промпте без изменения весов
Выравнивание	alignment	приведение поведения к предпочтениям и политикам
Инъекция промпта	prompt injection	подмена инструкции через недоверенный текст в контексте
Состязательный суффикс	adversarial suffix	оптимизированная строка токенов, снимающая отказ

Вопросы для самопроверки (часть 1)

1

Scaled dot-product attention

Выведите, почему стоит делитель $\sqrt{d_k}$, и что произойдёт при его удалении. Покажите, что self-attention без позиционного кодирования перестановочно инвариантно. Как каузальная маска частично восстанавливает информацию о порядке?

2

LSTM vs GRU

Сравните LSTM и GRU по числу параметров при одинаковой размерности состояния. Почему аддитивная структура $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ помогает градиенту?

3

KV-кэш

Посчитайте память KV-кэша для модели с $L = 32$, $H_{kv} = 8$, $d_{head} = 128$, fp16 при контексте 128k токенов. Во сколько раз её уменьшит переход к $H_{kv} = 2$?

4

FFN vs внимание

В каких режимах n и d доминирует стоимость FFN, а в каких — внимания? Чем RoPE принципиально отличается от обучаемых абсолютных позиций с точки зрения экстраполяции длины?

Вопросы для самопроверки (часть 2)

1

Линейное внимание и гибриды

Что именно линеаризуется в линейном внимании и какая способность при этом теряется? Почему гибридные архитектуры сохраняют небольшое число слоёв полного внимания вместо полного отказа от него?

2

МоЕ

Как соотносятся общие и активные параметры МоЕ и почему это не «бесплатное» увеличение ёмкости?

3

GCG и prompt injection

Сформулируйте задачу поиска состязательного суффикса как задачу дискретной оптимизации и объясните роль градиента в GCG. Почему успех prompt injection не устраняется усилением системного промпта?

4

Метрики и декодирование

Приведите три различия между физической атакой (лекция 6) и атакой на LLM: по каналу воздействия, типу пространства поиска и метрике успеха. Почему ASR нельзя сравнивать между работами без совпадения протокола? Как выбор параметров декодирования влияет на оценку устойчивости к jailbreak?

Приложения: полный проход модели и числовой пример

Приложение А. Полный проход decoder-only модели

Для входа $x = (x_1, \dots, x_n)$, словаря V , размерности d , L слоёв:

$$h^{(0)} = E[x], \quad E \in \mathbb{R}^{|V| \times d}$$

Для $\ell = 1..L$: $Q, K, V = \tilde{h} W^Q, W^K, W^V \rightarrow \text{MHA} \rightarrow \text{остаточная связь} \rightarrow \text{RMSNorm} \rightarrow \text{FFN} \rightarrow \text{остаточная связь} \rightarrow \text{RMSNorm}$. Число параметров плотной модели без эмбедингов приблизительно $12Ld^2$ при стандартных пропорциях (4 матрицы внимания $d \times d$ и FFN с расширением $4d$); при SwiGLU коэффициент немного иной за счёт третьей проекции.

Приложение В. Числовой пример внимания

Пусть $d_k = 2$, три токена, $Q = K = V$ заданы строками $q_1 = (1, 0)$, $q_2 = (0, 1)$, $q_3 = (1, 1)$.

Логиты $QK^T/\sqrt{2}$ для запроса q_3 : $(1 \cdot 1 + 1 \cdot 0)/\sqrt{2} = 0.707$, 0.707 , $(1 \cdot 0 + 1 \cdot 1)/\sqrt{2} = 0.707$, $(1 \cdot 1 + 1 \cdot 1)/\sqrt{2} = 1.414$.

Softmax: $\exp(0.707) = 2.028$, 2.028 , $\exp(1.414) = 4.113$; сумма 8.169 ; веса $(0.248, 0.248, 0.503)$.

Выход: $0.248 \cdot (1, 0) + 0.248 \cdot (0, 1) + 0.503 \cdot (1, 1) = \mathbf{(0.751, 0.751)}$.



Задание: пересчитать при каузальной маске для запроса q_2 и объяснить, почему третий токен не влияет на результат.

Литература

Основы последовательностей и внимания

- Sutskever et al. Sequence to Sequence Learning (2014)
- Bahdanau et al. Neural Machine Translation by Jointly Learning to Align and Translate (2015)
- Cho et al. Learning Phrase Representations using RNN Encoder-Decoder / GRU (2014)
- Vaswani et al. Attention Is All You Need (2017)
- Devlin et al. BERT (2019); Brown et al. GPT-3 (2020)

Компоненты современного стека

- Su et al. RoFormer: RoPE (2021); Press et al. ALiBi (2022)
- Zhang & Sennrich. RMSNorm (2019); Shazeer. SwiGLU (2020)
- Ainslie et al. GQA (2023); Shah et al. FlashAttention-3 (2024)
- Liu et al. Lost in the Middle (2023); Leviathan et al. Speculative Decoding (2023)

Разреженность и масштаб

- Fedus et al. Switch Transformers (2021); Jiang et al. Mixtral 8×7B (2024)
- DeepSeek-AI. DeepSeek-V2 (2024), DeepSeek-V3 (2024), DeepSeek-V3.2 (2025)

Рекуррентность, SSM и гибриды

- Gu & Dao. Mamba (2023); Dao & Gu. Mamba-2 / SSD (2024)
- Mamba-3 (2026, arXiv 2603.15569)
- Peng et al. RWKV (2023); Lieber et al. Jamba (2024)
- Behrouz et al. Titans (2025); Raschka S. Hybrid Attention обзор

Альтернативные парадигмы

- Nie et al. LLaDA (2025); Ye et al. Dream 7B (2025)
- Pagnoni et al. Byte Latent Transformer (ACL 2025)

Адаптация, выравнивание

- Ouyang et al. InstructGPT / RLHF (2022); Rafailov et al. DPO (2023)
- Hu et al. LoRA (2021); Wei et al. Chain-of-Thought (2022)
- DeepSeek-AI. DeepSeek-R1 (2025)
- Zou et al. GCG / Universal Adversarial Attacks (2023)
- Wei et al. Jailbroken (2023); Greshake et al. Indirect Prompt Injection (2023)
- Wallace et al. Instruction Hierarchy (2024); Ardit et al. Refusal Direction (2024)
- Anil et al. Many-shot Jailbreaking, Anthropic (2024)
- Land & Bartolo. Fishing for Magikarp (2024)